CASHEW PEST IDENTIFICATION AND CONTROL SYSTEM

(AN AI-POWERED MULTILINGUAL PLATFORM FOR SUSTAINABLE CASHEW CULTIVATION)

Cashew Pest Analyzer

Project Final Report

Usman Hussain

B.Sc. (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

I declare that this is my own work, and this thesis does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning. To the best of my knowledge and belief, it does not contain any material previously published or written by another person except where acknowledgement is made in the text. I also hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation, in whole or in part, in print, electronic, or other medium. I retain the right to use this content in whole or in part in future works such as articles or books.

Name	**Student ID**	**Signature**
Usman Hussain	IT22351104	

The above candidate is carrying out research for the undergraduate dissertation under my supervision.

…… …….

Signature of the supervisor Date

Ms. Shashika Lokuoliyana

…… ……

Signature of the co-supervisor Date

Ms.Suranjini Sliva

ABSTRACT

Agriculture is a cornerstone of Sri Lanka's economy, and cashew cultivation in particular plays a vital role in the livelihoods of rural farming communities. However, cashew production is continuously threatened by pest infestations, with three pests being especially destructive: the Cashew Stem Borer, Thrips, and Red Mites. The Cashew Stem Borer is particularly damaging because once it bores into the trunk, the affected tree can die within three months. Traditional pest identification depends entirely on the farmer's visual inspection, which is both time-consuming and unreliable, especially for small pests such as Thrips and Red Mites whose features are barely visible to the unaided eye. Delayed identification leads to delayed intervention, excessive use of pesticides, environmental damage, and significant yield loss.

This research proposes a Cashew Pest Identification and Control System that combines deep learning-based object detection with image processing to provide accurate, real-time pest detection on a mobile device. A YOLOv8 (Nano variant) object detection model was trained on a custom dataset that combined a publicly available dataset from Kaggle with field images collected directly from cashew plantations in Puttalam, Sri Lanka. The dataset was annotated and augmented using Roboflow. The trained model was converted to TensorFlow Lite for on-device inference and integrated into a Flutter mobile application. The application allows farmers to capture an image of an affected plant part, identifies the pest type, and immediately recommends both biological and chemical control measures.

A key contribution of this work is its accessibility. The mobile application supports three languages: English, Tamil, and Sinhala. This trilingual design directly addresses the language barriers faced by Sri Lankan farmers, many of whom are not fluent in English. Experimental evaluation showed that the system achieves a mean Average Precision (mAP@50) of 93.1%, a precision of 92.6%, a recall of 89.5%, and an estimated F1 score of 0.91, while maintaining a compact model size of approximately 6.2 MB suitable for low-end mobile devices. The proposed system reduces dependence on manual inspection, supports timely and informed pest management decisions, and contributes to sustainable cashew production in Sri Lanka.

Keywords: Cashew Pest Detection, YOLOv8, Object Detection, TensorFlow Lite, Multilingual Mobile Application, Flutter, Sustainable Agriculture, Sri Lanka.

ACKNOWLEDGEMENT

The completion of this research project would not have been possible without the support, guidance, and encouragement of many individuals and institutions, to whom I would like to express my sincere gratitude.

First and foremost, I would like to thank my supervisor and co-supervisor for their continuous guidance, valuable advice, and constructive feedback throughout the entire research project. Their expertise in the field and their patient mentoring were instrumental in shaping the direction of this work and overcoming the technical challenges encountered along the way.

I would also like to extend my heartfelt appreciation to all the lecturers, instructors, assistant lecturers, and both academic and non-academic staff of the Sri Lanka Institute of Information Technology (SLIIT) for their support during my undergraduate studies and during this research.

My sincere thanks go to the cashew farmers of Puttalam who generously allowed me to visit their fields and collect images for the dataset. Their willingness to share their experiences and observations greatly enhanced the practical relevance of this work.

I am also grateful to my fellow group members for their cooperation and teamwork, and to my family and friends, whose unwavering support and encouragement made this entire journey possible.

TABLE OF CONTENTS

Declaration i

Abstract ii

Acknowledgement iii

Table of Contents iv

List of Figures vi

List of Tables vi

List of Abbreviations vii

1. INTRODUCTION 1

1.1 Background Study and Literature Review 1

1.1.1 Background Study 1

1.2 Literature Review 4

1.3 Research Gap 6

1.4 Research Problems 8

1.5 Research Objectives 9

2. METHODOLOGY 11

2.1 Methodology 11

2.1.1 Agile Principles Applied in the Project 12

2.1.2 Feasibility Study and Planning 14

2.1.3 Requirement Gathering and Analysis 17

2.1.4 Designing the System 21

2.1.5 System Architecture Diagram 22

2.1.6 Implementation 24

2.1.7 Development Environment and Tools 26

2.1.8 Data Collection 27

2.1.9 Data Preprocessing and Augmentation 29

2.1.10 Model Training 31

2.1.11 How the Algorithms Work 34

2.1.12 Testing 37

2.2 Budget and Commercialization Aspects of the Project 41
3. RESULTS AND DISCUSSION 43
3.1 Results 43
3.2 Discussion 46
3.3 Future Scope 48
3.4 Conclusion 49
References 50

LIST OF FIGURES

Figure 1: Cashew Stem Borer damage to a cashew trunk 2
Figure 2: YOLOv8 architecture overview 3
Figure 3: Agile development life cycle 13
Figure 4: Gantt chart of the project schedule 16
Figure 5: Work breakdown structure 17
Figure 6: System architecture diagram 23
Figure 7: Mobile application interface (trilingual) 25
Figure 8: Dataset class distribution and bounding box statistics 28
Figure 9: Annotated training batch with mosaic augmentation applied 30
Figure 10: Training and validation curves across 50 epochs 33
Figure 11: Normalised confusion matrix on the validation set 38
Figure 12: Precision–recall curves per class and overall (mAP@50) 44
Figure 13: F1 score as a function of confidence threshold 45
Figure 14: Sample model predictions on a validation batch 46

LIST OF TABLES

Table 1: Comparative analysis of related research 7
Table 2: Use case for cashew farmer 20
Table 3: YOLOv8 training hyperparameters 32
Table 4: Per-class detection performance 37
Table 5: Test cases for system validation 39
Table 6: Project budget per month 42

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
AMP	Automatic Mixed Precision
AdamW	Adam with Decoupled Weight Decay (Optimizer)
API	Application Programming Interface
CNN	Convolutional Neural Network
DFL	Distribution Focal Loss
DL	Deep Learning
FPS	Frames Per Second
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value Colour Space
IoU	Intersection over Union
IPM	Integrated Pest Management
mAP	Mean Average Precision
ML	Machine Learning
MPS	Metal Performance Shaders (Apple Silicon GPU)
SGD	Stochastic Gradient Descent
SUS	System Usability Scale
TFLite	TensorFlow Lite
UI	User Interface
YOLO	You Only Look Once (Object Detection Family)
YOLOv8	You Only Look Once, Version 8

1. INTRODUCTION

1.1 Background Study and Literature Review

1.1.1 Background Study

Agriculture has played a defining role in Sri Lanka's economy and culture for centuries. Among the country's many agricultural sectors, cashew cultivation occupies a particularly important position. Cashew nuts are one of Sri Lanka's most valuable cash crops, providing both export revenue and a stable source of income for thousands of small- and medium-scale farmers, especially in the dry and intermediate zones such as the North-Western and Eastern Provinces. The crop is well-suited to the local climate and soil conditions, and a single mature cashew tree can produce nuts for several decades when properly maintained.

Despite its economic importance, cashew cultivation in Sri Lanka faces a number of persistent challenges. Among these, pest infestation stands out as one of the most damaging. Three pests in particular cause repeated and severe losses: the Cashew Stem Borer, Thrips, and Red Mites. The Cashew Stem Borer is widely regarded as the most destructive of the three. Once larvae penetrate the trunk of the tree, they tunnel through the vascular tissue, disrupting the flow of water and nutrients. Field experience and prior reports indicate that an infested tree can decline rapidly and die within as little as three months. Because the tree dies from the inside, external symptoms often appear only after extensive internal damage has already occurred.

Thrips and Red Mites, while less spectacular in their effects, also cause substantial yield reduction. Thrips feed on the underside of leaves and on flower panicles, leading to leaf curling, scarring of nuts, and reduced flowering. Red Mites are extremely small, often smaller than a pinhead, and they cluster on the undersides of leaves where they pierce plant cells and extract their contents. Their feeding produces a characteristic stippling and bronzing of leaves, weakening the plant and reducing photosynthesis. Both pests are notoriously difficult for farmers to identify with the naked eye because of their small size and cryptic appearance.

Figure 1 shows a cashew trunk from a Puttalam plantation exhibiting the characteristic damage signature of a Cashew Stem Borer infestation: the bark has split and peeled away, exposing reddened moist cambium tissue, and a dark entry cavity is visible at the junction of two major branches — a stage at which the tree's vascular system has already been severely compromised.

Figure 1: Cashew Stem Borer damage to a cashew trunk (puttalam, Sri Lanka)

Traditional pest identification in Sri Lankan cashew plantations relies almost entirely on visual inspection by the farmer. While experienced farmers may recognize obvious symptoms such as boreholes or wilting foliage, they often struggle to identify smaller pests at early stages of infestation. Identification is further complicated by the visual similarity between certain pests and harmless arthropods, and by the fact that early symptoms may overlap across different causes. As a result, farmers frequently take one of two undesirable actions: they wait until damage is severe and then apply broad-spectrum chemical pesticides, or they apply chemicals prophylactically without confirmed identification. Both responses lead to overuse of agrochemicals, environmental contamination, soil degradation, and increased production costs, while also encouraging the development of pesticide resistance in pest populations.

In recent years, advances in Artificial Intelligence (AI) and computer vision have begun to transform how agricultural problems are addressed. Convolutional Neural Networks (CNNs) and modern object detection architectures such as the You Only

Look Once (YOLO) family have demonstrated strong performance in tasks ranging from leaf disease classification to fruit counting and weed identification. Crucially, these models can now be compressed and deployed on commodity smartphones using frameworks such as TensorFlow Lite, making them accessible to ordinary farmers without the need for cloud connectivity or specialised equipment.

Figure 2 illustrates the YOLOv8 Nano architecture used in this project. The network is divided into three stages: a CSP-DarkNet backbone with C2f blocks that progressively downsample the input from 80×80 to 20×20 feature maps, a Feature Pyramid Network (FPN) and Path Aggregation Network (PAN) neck that fuses features across scales, and an anchor-free detection head that predicts at three spatial resolutions (P3/P4/P5). Final detections are produced after confidence thresholding and Non-Maximum Suppression (NMS).

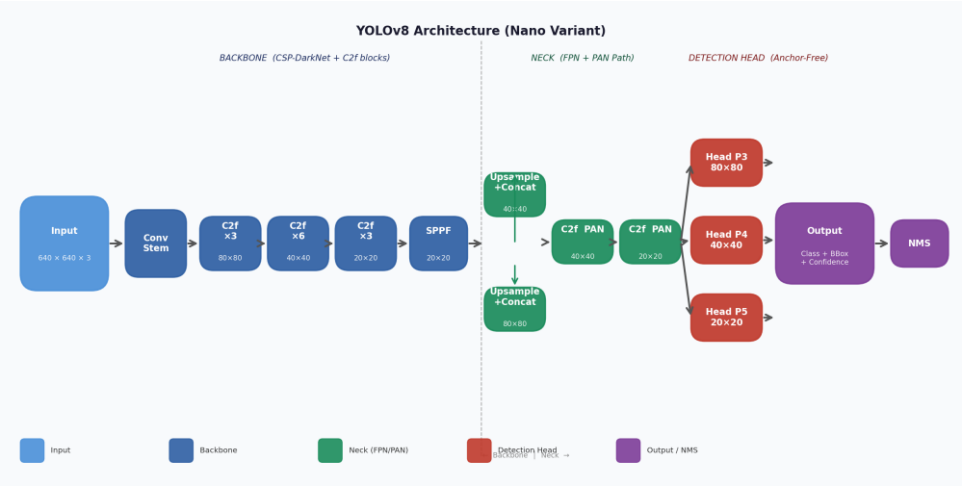

Figure 2: YOLOv8 Nano architecture — Backbone, Neck (FPN+PAN), and anchor-free Detection Head

The proposed Cashew Pest Identification and Control System builds on these advances. It applies a YOLOv8-based object detection model trained on cashew-specific imagery, deploys it on mobile devices through a Flutter application, and pairs each detection with biological and chemical control recommendations. A particular emphasis of the system is on accessibility: the application supports three languages — English, Tamil, and Sinhala — so that farmers in different regions of the country can use it comfortably in their first language. By bringing reliable pest identification and

treatment guidance directly into the hands of farmers, this work seeks to support timely intervention, reduce indiscriminate use of agrochemicals, and contribute to a more sustainable cashew industry in Sri Lanka.

1.2 Literature Review

The application of computer vision to agricultural pest detection has been an active research area for over a decade. Early studies relied on hand-crafted feature extraction, in which colour histograms, texture descriptors, and shape features were combined with classical machine learning classifiers such as Support Vector Machines or Random Forests. These approaches achieved reasonable performance on controlled datasets but struggled with the variability of real field images, where lighting, angle, scale, and background clutter differ widely from one capture to the next.

The introduction of deep convolutional neural networks (CNNs) marked a turning point. Krizhevsky et al. demonstrated that deep architectures, when trained on sufficiently large datasets, can learn rich hierarchical representations that generalise far better than hand-crafted features. Subsequent work by researchers such as He et al. on residual networks, and by Simonyan and Zisserman on the VGG family, established CNNs as the dominant approach for image classification. In the agricultural domain, Mohanty et al. applied transfer learning with pre-trained CNNs to classify plant diseases from leaf images and reported high accuracy on the PlantVillage dataset, sparking a wave of follow-up work in crop disease and pest classification.

While classification answers the question of "what is in the image", many real-world agricultural problems require object detection — locating and identifying specific objects within an image, possibly multiple instances per image. The YOLO family of detectors, introduced by Redmon et al., reformulated detection as a single regression problem and enabled real-time performance. Successive versions (YOLOv3, YOLOv4, YOLOv5, YOLOv7, and YOLOv8) have steadily improved both accuracy and speed, while also becoming increasingly accessible through well-maintained open-source implementations. YOLOv8, released by Ultralytics, introduced an anchor-free detection head, an improved feature pyramid, and a streamlined training

pipeline that makes transfer learning accessible to practitioners without specialised infrastructure.

Several studies have applied YOLO-based detectors to agricultural pests. For example, work on rice pest detection in paddy fields has demonstrated that small-object pests can be reliably detected when sufficient annotated data is available and when augmentation strategies such as mosaic and HSV jittering are used. Similar studies on tea, citrus, and tomato pests confirm that YOLO models trained on a few hundred to a few thousand annotated images can achieve mean Average Precision (mAP) values above 0.85 in field conditions. These findings motivated the choice of YOLOv8 for the present work.

Mobile deployment of pest detection models is another active research direction. Frameworks such as TensorFlow Lite and ONNX Runtime allow trained models to be quantised and run efficiently on smartphones with limited memory and compute capacity. Studies on TFLite-deployed plant disease classifiers report inference times of under one second per image on mid-range Android devices, which is more than adequate for interactive farmer-facing applications. Cross-platform mobile frameworks such as Flutter have further reduced the engineering effort needed to build a single application that runs on both Android and iOS.

In the Sri Lankan context, several research efforts have explored mobile and AI-driven solutions for paddy and tea cultivation. However, cashew cultivation remains comparatively underserved by digital tools. Existing local advisory applications are largely text-based and rule-based, providing static information but not real-time identification. Furthermore, very few systems offer multilingual interfaces; most assume English literacy, which excludes a significant portion of rural farmers. The literature therefore reveals a clear opportunity for a cashew-specific, image-based, multilingual decision support system that can operate on commodity smartphones in field conditions — which is precisely the gap addressed by this research.

1.3 Research Gap

Although AI-based pest detection is a well-studied problem internationally, several specific gaps remain when the focus is narrowed to cashew cultivation in Sri Lanka.

First, almost all published deep learning datasets and pre-trained models for pest detection target globally important commodity crops such as rice, wheat, maize, and tomato. Cashew, despite its economic importance in Sri Lanka, India, and parts of Africa, has received comparatively little attention. Publicly available cashew pest image datasets are small, inconsistently labelled, and often dominated by foreign cultivars and pests that do not match the species observed locally.

Second, existing mobile agricultural advisory applications available in Sri Lanka are predominantly informational. They provide reference images, descriptions of pests, and general treatment guidelines, but they typically lack on-device image-based identification. As a result, the burden of recognising the pest still falls entirely on the farmer. There is little integration between automatic identification and actionable recommendations.

Third, the language barrier is rarely addressed in academic prototypes. Most research applications are demonstrated in English, even when targeted at users who speak Tamil or Sinhala in their daily lives. This significantly limits real-world adoption. A practically usable system must communicate with farmers in a language they fully understand, especially when conveying chemical dosage instructions or biological control techniques where misunderstanding can be hazardous.

Fourth, cashew-specific recommendations integrating both biological and chemical control are not commonly bundled with detection in existing tools. Sustainable agriculture increasingly favours Integrated Pest Management (IPM), in which biological controls are preferred when severity is low, and chemical interventions are reserved for cases where damage is severe or biological methods alone are insufficient. A system that automatically maps a detected pest to an IPM-aligned recommendation in the farmer's language would be a meaningful contribution.

Table 1 summarises how the proposed system compares with representative prior work on related problems.

Feature / Capability	Ref. [1]	Ref. [2]	Ref. [3]	Ref. [4]	Proposed
Cashew-specific dataset	✗	✗	✗	✗	✓
Object detection (not just classification)	✓	✗	✓	✗	✓
Real-time mobile inference	✗	✓	✗	✓	✓
Trilingual interface (English / Tamil / Sinhala)	✗	✗	✗	✗	✓
Combined biological + chemical advice	✗	✗	✗	✓	✓
Field images from Sri Lanka	✗	✗	✗	✗	✓

Table 1: Comparative analysis of related research

1.4 Research Problems

Cashew cultivation in Sri Lanka, particularly in regions such as Puttalam where the crop is grown extensively by smallholder farmers, is constantly threatened by infestations of the Cashew Stem Borer, Thrips, and Red Mites. The current approach to pest management in these communities is predominantly reactive and manual, and suffers from a number of well-documented weaknesses.

- **Inaccurate identification:** Thrips and Red Mites are extremely small and difficult to identify visually. The Cashew Stem Borer often causes internal damage long before external symptoms are obvious. Farmers without specialised training frequently misidentify pests or fail to detect infestation at early stages.
- **Delayed intervention:** Once symptoms become obvious, the damage is often advanced. In the case of the Cashew Stem Borer, an infested tree can decline within three months, and trees lost in this way require many years to be replaced economically.

- **Overuse of chemicals:** When in doubt, farmers tend to apply broad-spectrum pesticides as a precaution. This raises production costs, harms beneficial insects and pollinators, contaminates soil and water, and can ultimately reduce yields by killing natural pest enemies.
- **Language barriers:** A significant share of farmers in cashew-growing areas are most comfortable in Tamil or Sinhala. Existing English-only advisory tools are therefore underused, even when available.
- **Lack of integrated decision support:** Even when farmers correctly identify a pest, they may not have immediate access to up-to-date, IPM-aligned recommendations covering both biological and chemical control.

These observations lead to the central research problem of this thesis:

"How can a deep learning-based pest identification and recommendation system be designed and deployed on a mobile device so as to enable Sri Lankan cashew farmers to detect Cashew Stem Borer, Thrips, and Red Mites accurately and in real time, and to receive actionable, multilingual control recommendations that promote sustainable cultivation?"

This question motivates an integrated solution that combines reliable on-device computer vision, a usable mobile interface in the farmer's preferred language, and a curated knowledge base of biological and chemical control measures.

1.5 Research Objectives

Main Objective

To design, develop, and evaluate an AI-powered Cashew Pest Identification and Control System that performs real-time pest detection on a mobile device and provides multilingual biological and chemical control recommendations to support sustainable cashew cultivation in Sri Lanka.

Specific Objectives

1. To collect and curate a custom cashew pest dataset comprising images of Cashew Stem Borer, Thrips, and Red Mites, drawing on both publicly available sources (Kaggle) and original field photographs taken in Puttalam.
2. To annotate and augment the dataset using Roboflow so that the resulting training set is balanced, well-labelled, and robust to variations in lighting and orientation.
3. To train a YOLOv8 (Nano variant) object detection model on this dataset and evaluate its performance using standard detection metrics (mAP@50, mAP@50–95, precision, recall, F1).
4. To convert the trained model into TensorFlow Lite format suitable for on-device inference on commodity Android smartphones.
5. To develop a Flutter mobile application that integrates the TFLite model, captures images, displays detection results, and presents biological and chemical recommendations in English, Tamil, and Sinhala.
6. To validate the end-to-end system through technical testing, manual test cases, and a usability evaluation with representative cashew farmers.

Business Objectives

7. To reduce yield losses caused by undetected Cashew Stem Borer, Thrips, and Red Mites, thereby improving farmer income.
8. To reduce the unnecessary use of chemical pesticides by encouraging biological control as a first line of defence where appropriate.
9. To deliver a low-cost mobile solution accessible to smallholder farmers, requiring only a basic smartphone.
10. To promote the adoption of digital tools in Sri Lankan cashew cultivation as part of broader agricultural modernisation.

2. METHODOLOGY

2.1 Methodology

The development of the Cashew Pest Identification and Control System followed a structured, end-to-end methodology covering data collection, preprocessing, model training, system integration, mobile deployment, and evaluation. Each stage was treated as a discrete piece of work with its own goals and verification criteria, while still being integrated into a single iterative development cycle.

The project began with data collection. Three target classes were defined based on field consultations and on the literature: Cashew Stem Borer, Thrips, and Red Mites. A baseline dataset of cashew pest imagery was sourced from Kaggle, providing variety in pest appearance, lighting, and background. To strengthen the dataset's relevance to Sri Lankan field conditions, additional images were captured in person at cashew plantations in Puttalam using a smartphone camera. These field images proved especially valuable because they reflected the lighting, foliage, and cultivar that the deployed system would actually encounter.

All images were uploaded to Roboflow, where bounding box annotations were drawn around each instance of a target pest. Roboflow was also used to apply preprocessing (auto-orient, resize) and a controlled set of data augmentations (flipping, rotation, exposure changes, blur, and noise) so that each original image yielded multiple augmented training samples. The dataset was split into training, validation, and testing subsets, and exported in YOLO format for direct compatibility with the YOLOv8 training pipeline.

Model development used the Ultralytics YOLOv8 implementation, specifically the Nano variant (yolov8n) — chosen because of its strong accuracy-to-size ratio and its suitability for mobile deployment. Transfer learning from COCO-pretrained weights was used to accelerate convergence. Training was performed for 50 epochs at an image size of 640 × 640, with a batch size of 16, on Apple's Metal Performance Shaders (MPS) backend. Automatic Mixed Precision was enabled to reduce memory consumption and speed up training. The mosaic augmentation, which is built into the YOLOv8 pipeline, was disabled for the final 10 epochs (close_mosaic=10) to allow

the model to converge on cleaner, non-mosaicked representations — a recommended practice that produced a clear final-stage gain in mAP.

After training, the best checkpoint was exported to TensorFlow Lite. The TFLite model was then integrated into a Flutter mobile application that runs on Android (and is portable to iOS). Inference is performed entirely on-device, so the application does not depend on internet connectivity for pest identification. After detection, the application looks up the corresponding biological and chemical control recommendations and displays them in the farmer's selected language. A clean, large-button user interface was designed to suit users who may have limited experience with smartphones.

Evaluation involved both quantitative and qualitative components. Quantitatively, the trained model was assessed using mean Average Precision (mAP@50 and mAP@50–95), precision, recall, and F1 score on the held-out validation set, with confusion matrices and per-class precision–recall curves analysed for diagnostic insight. Qualitatively, the integrated mobile application was tested in controlled scenarios using a defined set of test cases, and a System Usability Scale (SUS) questionnaire was administered to a small group of cashew farmers from Puttalam to assess practical usability.

2.1.1 Agile Principles Applied in the Project

Given the exploratory nature of applied machine learning research and the need to integrate diverse components — data, models, and a mobile front end — an Agile-inspired development approach was adopted. The project was organised into a sequence of short sprints, each ending in a working artefact that could be reviewed, evaluated, and iterated upon.

Adaptive planning was central. Initial sprints focused on dataset preparation and on training a baseline YOLOv8 model. As soon as baseline results were available, plans for the subsequent sprints were updated based on observed weaknesses — for example, when early results showed reduced recall on Red Mites, the next sprint was redirected to gather additional Red Mite imagery and to revisit augmentation settings. This

avoided the trap of over-committing to a single training plan that might not match the data.

Continuous improvement was practised through repeated training–evaluation cycles. Each model checkpoint was evaluated with mAP, precision, recall, and a per-class confusion matrix; the resulting feedback informed adjustments to data balance, augmentation strength, and training schedule. Because YOLOv8 makes such iterations cheap, several variants of the model could be compared before committing to the final deployed checkpoint.

Risk management was treated as an ongoing concern rather than a one-time checklist. Identified risks included dataset class imbalance, mismatch between Kaggle imagery and Sri Lankan field imagery, mobile inference latency, and language localisation errors. Each risk was tracked, mitigated where possible (for example, by augmenting with field photos from Puttalam, by quantising the TFLite model, and by reviewing translations with native speakers), and re-evaluated at each sprint boundary.

Collaboration with stakeholders was maintained throughout. Cashew farmers in Puttalam were consulted both at the start of the project (during requirements gathering and field photography) and during testing. Their feedback influenced practical design choices such as the size of buttons in the mobile interface, the wording of recommendations, and the inclusion of a confidence indicator on each detection result.

Figure 3 illustrates the Agile development lifecycle as applied to this project. Each sprint moves clockwise through six phases — Plan, Design, Develop, Test, Deploy, and Review — with the sprint retrospective feeding back into the next planning cycle. Iteration lengths were kept to one to two weeks throughout the project.

Figure 3: Agile development lifecycle applied to the Cashew Pest Identification System

2.1.2 Feasibility Study and Planning

Before committing to development, a feasibility study was conducted to ensure that the project could realistically be completed within the available time, budget, and skills. The study considered four dimensions: technical, economic, operational, and scheduling feasibility.

Technical feasibility was favourable. The required components — a deep learning framework (PyTorch through Ultralytics YOLOv8), a model conversion path to TensorFlow Lite, a cross-platform mobile framework (Flutter), and a localisation pipeline — are all mature, open-source, and well-documented. The training hardware required was modest: a developer laptop with Apple's MPS GPU was sufficient to train the YOLOv8 Nano variant within a few hours per run. Inference hardware was even lighter: any Android smartphone with at least 2 GB of RAM can run the quantised TFLite model in under a second per image.

Economic feasibility was also strong. The project relies almost entirely on free and open-source software. The largest material cost was travel for field data collection in Puttalam. No commercial datasets, paid APIs, or proprietary tools were used in the

production pipeline. The resulting system can therefore be distributed to farmers at zero or near-zero cost, which is essential for adoption among smallholders.

Operational feasibility considered whether the eventual system would actually be usable by its target audience. The decision to provide a trilingual interface (English, Tamil, Sinhala), the choice of a smartphone-based delivery channel rather than a desktop tool, and the simple capture-and-show workflow were all motivated by operational considerations. Field discussions with farmers in Puttalam confirmed that smartphones are widely available and that on-device, internet-free operation is a strong selling point given inconsistent rural connectivity.

Scheduling feasibility was managed by breaking the project into clearly defined phases — planning and research, data collection, data preprocessing, model training, integration with control suggestions, app integration and UI design, final testing and refinement, and launch — each placed on a Gantt chart with explicit start and end dates. The Agile sprint structure provided a built-in mechanism for absorbing schedule risk: if a sprint ran long, lower-priority items could be deferred without endangering the overall delivery.

The project was scheduled over twelve months from June through May, spanning all phases from initial planning to deployment. Figure 4 shows the full Gantt chart. Key milestones include the proposal presentation (August), software design and architectural design (October–November), implementation and front-end development (November–February), integration and system testing (December–May), and the final presentation, documentation, and system deployment (April–May).

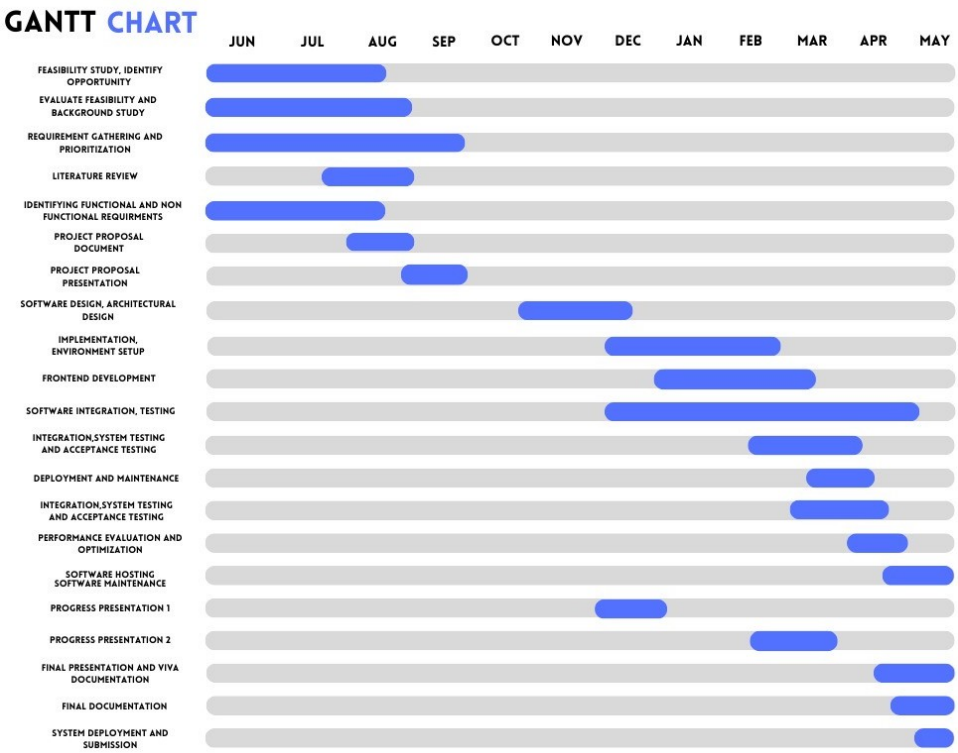

Figure 4: Gantt chart of the project schedule (June – May)

Figure 5 presents the Work Breakdown Structure (WBS), decomposing the project into five top-level work packages: Planning (design scope, timeline, pre-assignment, proposal, charter), Analysis (requirement gathering, understanding old systems, eliminating error), Design (application architecture, interface design, database design), Implementation (collect dataset, create model, train model, test model), and Launch (advertise the product).

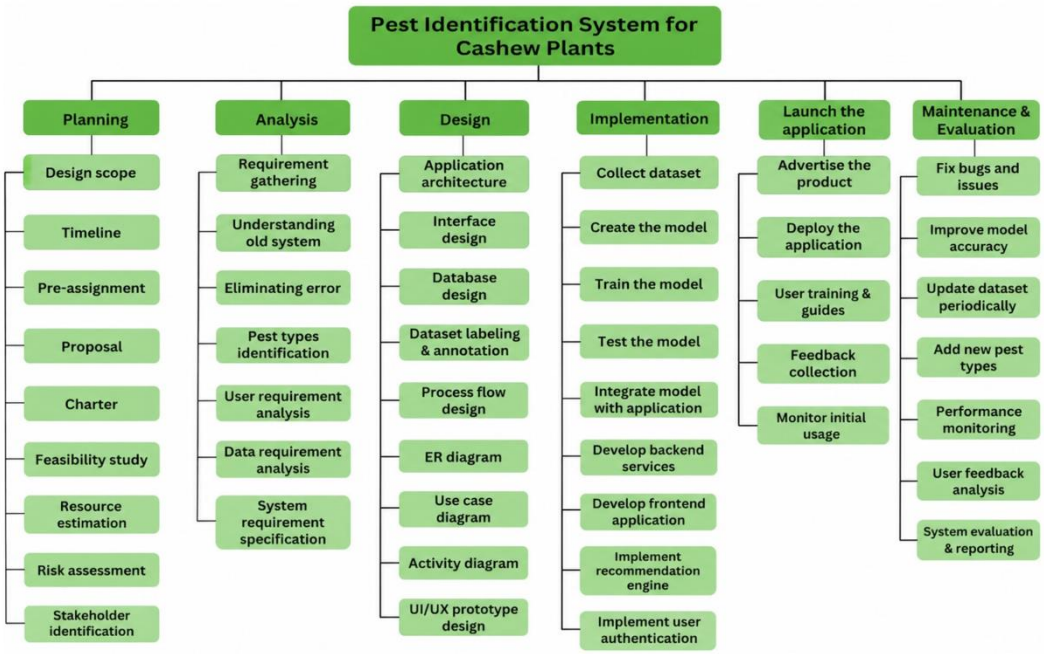

Figure 5: Work breakdown structure

2.1.3 Requirement Gathering and Analysis

Requirements for the system were gathered through informal interviews with cashew farmers in Puttalam, supplemented by a review of agricultural extension materials and academic literature. The requirements were then organised into functional, non-functional, user, system, and hardware categories.

Functional Requirements

- The system must allow the user to capture an image directly from the smartphone camera or upload an image from the gallery.
- The system must run an on-device YOLOv8 detector to identify Cashew Stem Borer, Thrips, and Red Mites in the captured image, drawing bounding boxes around each detected pest.
- The system must report the predicted pest class, the bounding box, and the model's confidence score for each detection.

- Based on the detected pest, the system must display both biological and chemical control recommendations in the farmer's selected language.
- The system must allow the user to switch between English, Tamil, and Sinhala at any time without restarting the application.
- The system must operate fully offline once installed; an active internet connection must not be required for detection or for viewing recommendations.
- The system must allow the user to view a history of past detections on the device.

Non-Functional Requirements

- Detection accuracy on the held-out validation set must achieve at least mAP@50 of 0.85 and recall of at least 0.85, so that the deployed model is genuinely useful in the field.
- End-to-end response time on a mid-range Android device must remain under three seconds from image capture to displayed recommendation.
- The user interface must be usable by farmers with limited smartphone experience, using large buttons, simple icons, and minimal text on the main screens.
- All translations must be reviewed by native speakers of Tamil and Sinhala before release.
- The application must run on Android 8.0 (Oreo) or newer, which covers the vast majority of devices in use in rural Sri Lanka.

User Requirements

- Farmers want a tool that gives them an immediate, trustworthy answer when they spot something unusual on their cashew trees, without sending data to the cloud or relying on connectivity.
- Farmers want guidance in their preferred language, with chemical dosage instructions stated clearly.

- Farmers prefer biological control suggestions when these are likely to be effective, and clear escalation to chemical control only when severity demands it.

System Requirements

- Backend training infrastructure: Python 3.10+, PyTorch with the Ultralytics YOLOv8 package, OpenCV, and Roboflow's annotation/export pipeline.
- Conversion pipeline: YOLOv8's built-in export to TensorFlow Lite (FP16 or INT8 quantisation, as appropriate).
- Mobile platform: Flutter 3.x, with the tflite_flutter and image packages for on-device inference and image preprocessing.
- Localisation: Flutter's intl package, with translation strings stored in versioned ARB files.

Hardware Requirements

- Development workstation: a laptop with at least 16 GB of RAM and a GPU (CUDA-capable NVIDIA GPU, or Apple Silicon with MPS) for training the YOLOv8 model.
- Target user device: any Android smartphone running Android 8.0 or newer, with at least 2 GB of RAM, a rear camera of 5 MP or higher, and 100 MB of free storage for the application and the model.

Use Case

Use Case ID	UC-01
Use Case Name	Cashew Pest Detection and Recommendation
Preconditions	1. The application is installed on the farmer's smartphone. 2. The smartphone has a working rear camera.
Primary Actor	Cashew Farmer (e.g. a smallholder in Puttalam)
Main Success Scenario	1. The farmer opens the application and selects a language. 2. The farmer captures an image of an affected leaf, twig, or trunk. 3. The application preprocesses the image (resize, normalise). 4. The on-device YOLOv8 TFLite model runs inference and

	returns detections with bounding boxes. 5. The application displays the detected pest(s) with confidence scores. 6. The application displays both biological and chemical control recommendations in the selected language. 7. The detection is saved to local history.
Extensions	4a. If no pest is detected with sufficient confidence, the application advises the farmer to retake the photo with better lighting / focus. 6a. The farmer can switch language at any point and recommendations re-render.

Table 2: Use case for cashew farmer

User Stories

User Story 1 — Cashew Farmer: As a cashew farmer, I want to take a photo of an affected part of my tree and immediately learn which pest is present and how to treat it in my own language, so that I can act before the infestation spreads.

Acceptance criteria: the application identifies Cashew Stem Borer, Thrips, or Red Mites with high confidence; recommendations are shown in English, Tamil, or Sinhala as the farmer prefers; the entire workflow works without an internet connection.

User Story 2 — Agricultural Extension Officer: As an extension officer, I want to use the same tool with farmers during field visits so that I can verify their observations and provide consistent, IPM-aligned advice.

Acceptance criteria: the officer can switch languages quickly, share the result with the farmer, and view a brief explanation of why each recommendation is given.

2.1.4 Designing the System

The system was designed around three principles: trustworthiness, simplicity, and accessibility. Trustworthiness drove the choice of an object detection architecture that explicitly localises pests rather than merely classifying images, so that farmers can see exactly what the model has identified and where. Simplicity drove the design of a workflow with as few taps as possible: open the app, capture an image, see the result. Accessibility drove the trilingual interface and the offline-first architecture.

Internally, the system was decomposed into modular components: an image acquisition module, a preprocessing module, a YOLOv8 inference module, a

postprocessing module that filters detections by confidence and removes overlapping boxes via Non-Maximum Suppression, a recommendation module that maps pest classes to IPM-aligned advice, and a localisation module that retrieves the appropriate translations. This modularity makes the system easy to maintain and extend — for example, supporting an additional pest in future would require only retraining the model and adding new entries in the recommendation and translation tables.

2.1.5 System Architecture Diagram

The system architecture follows a layered design that maps closely onto the modular decomposition above.

At the top sits the user interface layer, implemented in Flutter. This layer handles language selection, camera capture, gallery import, and the rendering of detection results and recommendations.

Below the UI is the inference layer, which loads the YOLOv8 TFLite model at application start-up and exposes a simple synchronous interface (image in, list of detections out). This layer also performs the necessary preprocessing (BGR to RGB conversion if needed, resizing to 640 × 640, normalisation to the [0, 1] range) and postprocessing (decoding the raw output tensor into class labels, confidence scores, and bounding boxes; applying confidence thresholding and Non-Maximum Suppression).

Below the inference layer is the knowledge layer. This layer contains the static recommendation database — a mapping from each pest class to a structured set of biological and chemical recommendations, including dosages, target life stages, and safety notes. It also contains the localisation files for English, Tamil, and Sinhala.

Finally, a thin local persistence layer stores the user's detection history on-device, so that farmers can review past detections without an internet connection. No personal data leaves the device.

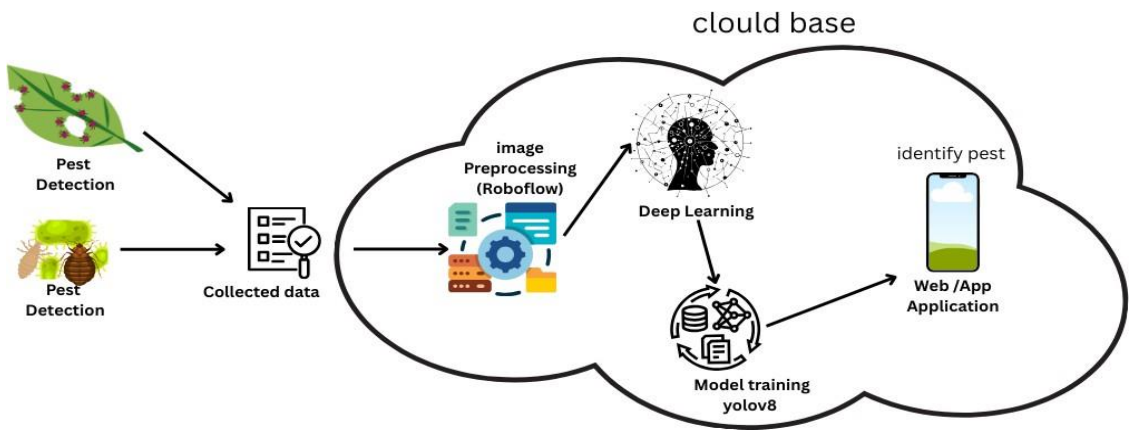

Figure 6: System architecture diagram

2.1.6 Implementation

Implementation followed the architectural plan and was carried out incrementally, with end-to-end working software available from very early in the project.

Model training was implemented in Python using the Ultralytics YOLOv8 package. A configuration file (cashew_pests.yaml) defined the dataset paths and the three classes. Training was launched with explicit hyperparameters: 50 epochs, image size 640, batch size 16, AMP enabled, close_mosaic set to 10. The best checkpoint by mAP@50 was retained automatically by the framework. Training artefacts — including loss curves, precision–recall curves, F1 vs confidence curves, and confusion matrices — were generated for each run and used to drive iterative improvements.

Model conversion to TensorFlow Lite was performed using YOLOv8's built-in export functionality, producing a quantised TFLite model approximately 6.2 MB in size. This quantisation step is critical for mobile deployment, since it reduces both file size and inference latency without materially affecting accuracy.

Mobile development was carried out in Flutter. The tflite_flutter package was used to load the TFLite model and run inference. Image preprocessing — including reading from camera or gallery, resizing, and normalising — was implemented using the image

package. Postprocessing was implemented in pure Dart, including confidence thresholding (0.25 by default) and class-aware Non-Maximum Suppression with an IoU threshold of 0.7.

The recommendation database was implemented as a structured JSON resource bundled with the application. Each entry contains the pest class, a short human-readable description, a list of biological control recommendations (e.g. natural predators, neem-based formulations), and a list of chemical control recommendations (e.g. specific active ingredients, dosages, and safety precautions). Localisation strings for each entry were maintained in Flutter's standard ARB format and translated by native speakers of Tamil and Sinhala. The user interface uses Flutter's MaterialApp localisation hooks to switch language at runtime.

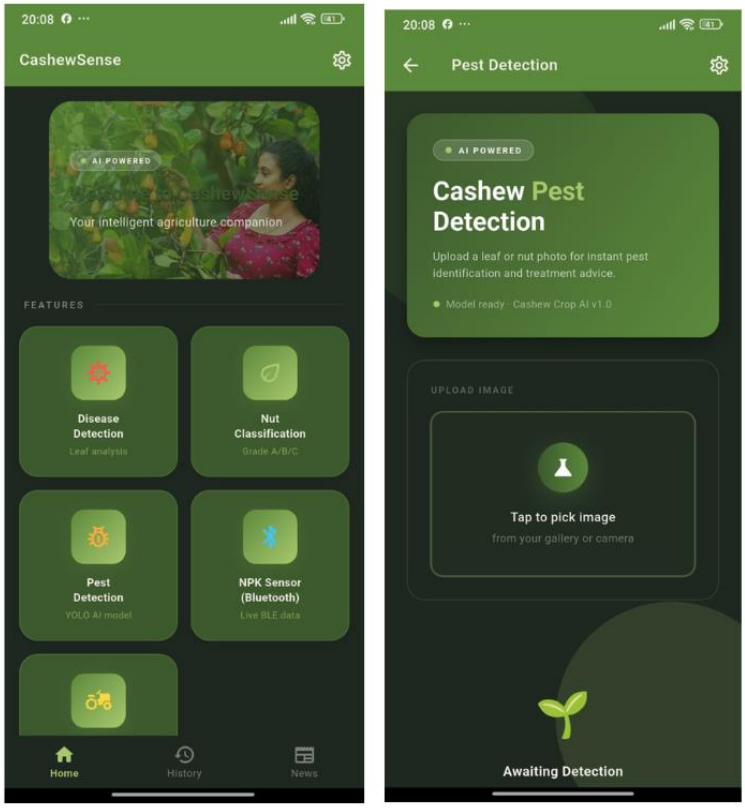

Figure 7: Mobile application interface (trilingual)

2.1.7 Development Environment and Tools

The development environment was deliberately kept lightweight and open-source. The training environment ran on a developer laptop with Python 3.10 and Ultralytics YOLOv8, using PyTorch with the MPS backend on Apple Silicon. OpenCV was used for ad-hoc image inspection. Roboflow's web interface was used for dataset annotation, augmentation, and export. Training experiments were tracked manually through the YOLOv8 results.csv file, with periodic visual review of the training curves and confusion matrices.

Mobile development used Flutter 3.x with the Dart 3.x toolchain. The tflite_flutter, image, camera, and intl packages were the core mobile dependencies. Android testing was carried out on a mid-range smartphone, which is broadly representative of devices in use in cashew-growing communities in Sri Lanka.

Version control was provided by Git. Project documentation, including this report and the user-facing recommendation content, was managed in Markdown so that it could be reviewed and edited collaboratively.

2.1.8 Data Collection

Data collection began with a survey of publicly available cashew pest imagery. A baseline dataset was downloaded from Kaggle, providing a useful starting point but suffering from two limitations: the imagery was dominated by foreign cultivars and locations, and the class balance for the three target pests was uneven, with Red Mites being noticeably under-represented.

To address these limitations, original field photographs were collected at cashew plantations in Puttalam, Sri Lanka. Images were captured with a smartphone camera under varied natural lighting conditions, including bright midday sun, overcast afternoons, and partial shade beneath the canopy. Where pests were not directly visible, characteristic damage symptoms (boreholes for the Cashew Stem Borer, leaf curling for Thrips, fine stippling for Red Mites) were also photographed to provide diverse contextual examples.

After cleaning and annotation, the final training dataset contained 1,479 annotated pest instances across the three target classes: 636 instances of Stem Borer, 491 instances of Thrips, and 352 instances of Mites. Stem Borer is over-represented because larvae and exit holes were photographically more accessible during field visits, while Mites are under-represented owing to their very small size and the difficulty of capturing them clearly in field conditions. Each image was reviewed before annotation to remove duplicates, blurry shots, and images where no target pest or symptom was clearly identifiable. Metadata such as the location, date, and lighting condition were recorded for a subset of the images to support later analysis.

Figure 8 summarises the resulting dataset: the top-left bar chart shows the per-class instance count, the top-right plot superimposes all annotated bounding boxes (showing that the dataset covers a wide range of object sizes and aspect ratios), and the lower plots show the spatial distribution of annotations across the image plane and the joint distribution of bounding box widths and heights.

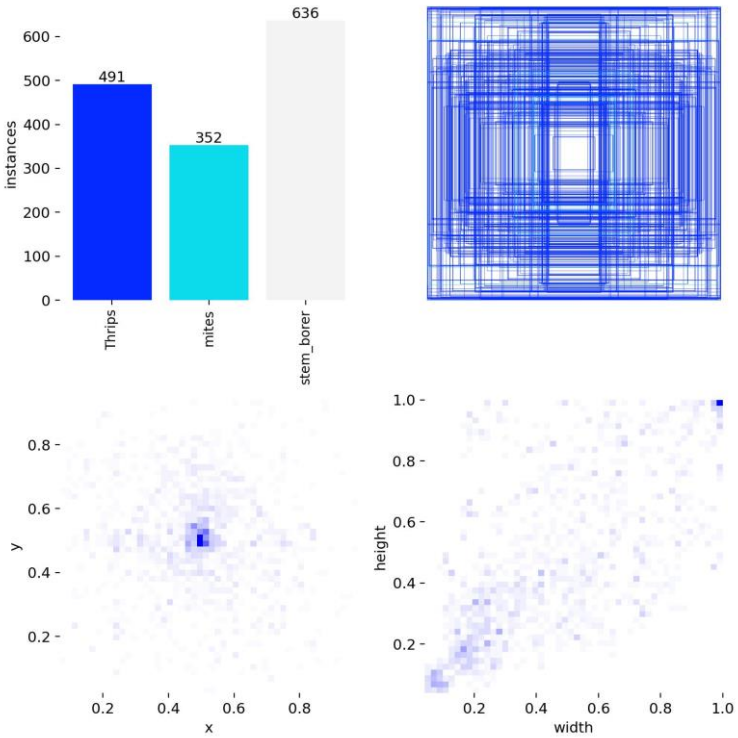

Figure 8: Dataset class distribution and bounding box statistics

2.1.9 Data Preprocessing and Augmentation

All collected images were uploaded to Roboflow, where bounding box annotations were drawn for every visible instance of Cashew Stem Borer, Thrips, or Red Mites. Annotation was performed carefully, paying particular attention to small instances of Thrips and Red Mites, which can easily be overlooked. Each image was assigned a single class label per box, and overlapping or ambiguous instances were resolved by re-photographing or by excluding the image.

Roboflow's preprocessing pipeline was configured to auto-orient images and resize them to 640 × 640 pixels — the input size expected by the YOLOv8 model. Pixel values were normalised to the [0, 1] range before being passed to the model.

Data augmentation was applied via Roboflow's augmentation pipeline so that each original training image yielded several augmented versions. The augmentations were chosen to mirror the conditions in which the deployed model would actually operate. Specifically, horizontal flipping (50% probability) accounts for left-right symmetry; HSV jittering (hue 0.015, saturation 0.7, value 0.4) simulates differences in time of

day, weather, and white balance; random scaling at 0.5 supports robustness to varied object sizes; mosaic augmentation (a YOLOv8 default) packs four training images into a single composite, helping the model learn to detect small objects in complex contexts; random erasing at 0.4 simulates partial occlusion by leaves or twigs; and small random translations (0.1) help the model handle imperfect framing.

Mosaic augmentation was disabled for the final 10 epochs of training (close_mosaic=10) so that the last phase of training would use clean, real-distribution-like images rather than mosaicked composites. This matches the recommended Ultralytics YOLOv8 training schedule and produced a noticeable final-stage improvement in mAP. Figure 9 shows a single training batch produced by the YOLOv8 data loader: each composite image is a 2 × 2 mosaic of four annotated training samples, and ground-truth bounding boxes are drawn on top with their class labels.

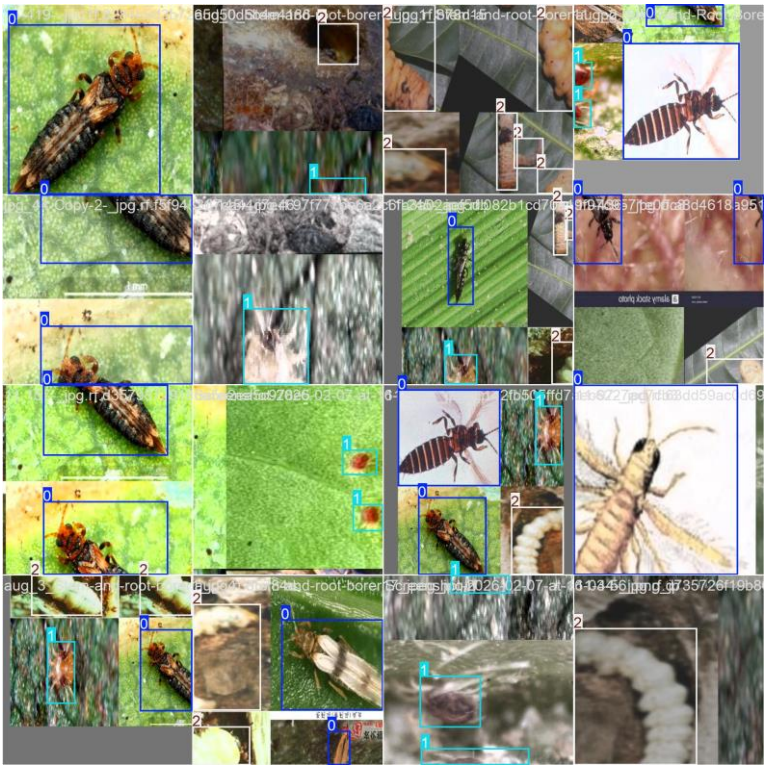

Figure 9: Annotated training batch with mosaic augmentation applied

2.1.10 Model Training

Training used the Ultralytics YOLOv8 Nano variant (yolov8n) initialised from COCO-pretrained weights. YOLOv8 was selected for several reasons. First, it is highly effective at detecting small objects, which is critical for Thrips and Red Mites. Second, it has a mature open-source implementation with strong documentation. Third, the Nano variant produces a compact model (≈6.2 MB after TFLite conversion) that is suitable for on-device inference. YOLOv11 was considered but not adopted; at the time of the project, YOLOv11's documentation was significantly less complete than YOLOv8's, which would have introduced unnecessary risk.

Training was performed for 50 epochs with the hyperparameters summarised in Table 3.

Parameter	Value
Architecture	YOLOv8 Nano (yolov8n)
Task	Object detection (3 classes)

Parameter	Value
Dataset configuration	cashew_pests.yaml
Epochs	50
Batch size	16
Image size	640 × 640
Device	Apple MPS (Metal Performance Shaders)
Optimizer	Auto (AdamW / SGD as selected by YOLOv8)
Mixed precision (AMP)	Enabled
Pretrained weights	Yes (COCO transfer learning)
Confidence threshold	0.25
IoU threshold	0.70
Mosaic augmentation	Enabled, disabled for final 10 epochs
Early stopping patience	100 (effectively disabled for full run)

Table 3: YOLOv8 training hyperparameters

Training proceeded in three observable phases. During the first phase (epochs 1 to roughly 10), the model learned the broad characteristics of each class; mAP@50 climbed rapidly from low values to the 0.77–0.85 range as the network adapted from its COCO initialisation. During the second phase (epochs 10 to 30), training stabilised: both training and validation losses decreased steadily, mAP@50 settled into the 0.84–0.89 range, and precision climbed past 0.90.

The third phase (epochs 40 to 50) coincided with the disabling of mosaic augmentation. Box loss dropped sharply (from approximately 0.81 to 0.68), classification loss dropped from 0.61 to 0.45, and mAP@50 jumped from around 0.88 to a peak of 0.9312 at epoch 46. This jump is consistent with documented YOLOv8 behaviour: the model converges on a cleaner representation once mosaic augmentation no longer perturbs the training distribution. The best checkpoint (best.pt) was retained at epoch 46 with mAP@50 = 0.9312, mAP@50–95 = 0.7020, precision = 0.9256, and recall = 0.8949. The final epoch (50) showed slightly different trade-offs (higher

precision, slightly lower recall) but the mAP@50-optimal checkpoint at epoch 46 was selected for deployment.

During training, a brief pause and resumption was recorded between epochs 43 and 44 (cumulative training time reset). This had no observable effect on convergence; the loss and mAP curves continued smoothly across the discontinuity. Figure 10 shows the full training dashboard. The top row presents the three training losses (box, classification, and DFL) alongside precision and recall on the validation set; the bottom row presents the corresponding validation losses and the two mAP metrics (mAP@50 and mAP@50–95). The sharp inflection visible in every panel near epoch 40 corresponds to the close_mosaic transition described above.

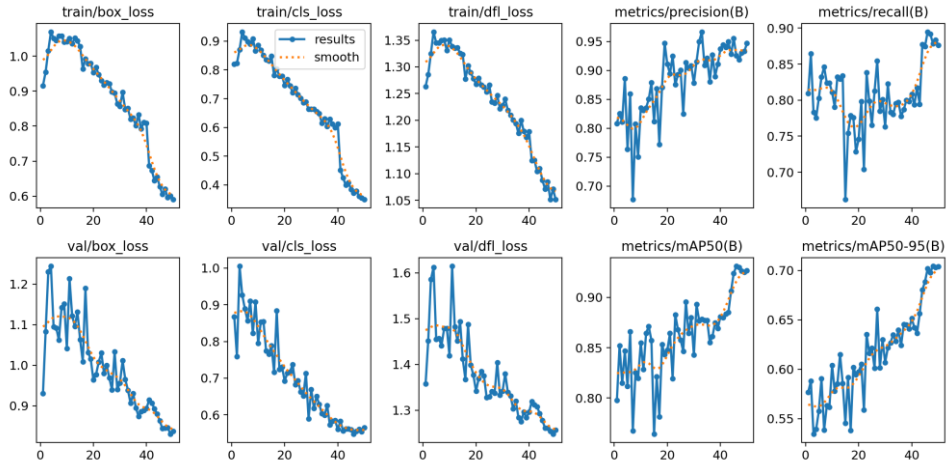

Figure 10: Training and validation curves across 50 epochs

2.1.11 How the Algorithms Work

The detection pipeline consists of a single trained neural network whose internal operation can be summarised in three stages: feature extraction, multi-scale prediction, and postprocessing.

In the feature extraction stage, the input image (resized to 640 × 640 and normalised) passes through YOLOv8's backbone, a convolutional network that produces feature maps at several spatial resolutions. The early layers respond to low-level cues such as edges and colour blobs, while deeper layers respond to higher-level concepts such as the elongated body of a stem borer larva or the clustered patterns of a mite colony.

YOLOv8's backbone is designed to be lightweight in the Nano variant, which is what allows the resulting model to run on mobile devices.

In the multi-scale prediction stage, the backbone's feature maps are passed through a feature pyramid network (FPN) that combines high-resolution, semantically shallow features with low-resolution, semantically rich features. This fusion is what gives YOLOv8 its strong small-object performance: information about small pests is preserved by the high-resolution branches, while the low-resolution branches contribute the contextual knowledge needed to disambiguate them. An anchor-free detection head then predicts, at each spatial location and scale, a class distribution and a bounding box offset, plus a confidence score.

In the postprocessing stage, raw predictions are filtered by confidence (using the configured threshold of 0.25) and reduced by class-aware Non-Maximum Suppression (with the IoU threshold set to 0.70). The result is a small list of bounding boxes, each with a class label and a confidence score, that constitute the final detections shown to the user.

On the recommendation side, a deterministic lookup is performed: each detected class is mapped to its corresponding biological and chemical recommendations in the knowledge base, and the appropriate translations are retrieved through Flutter's localisation system. Where multiple classes are detected in the same image, recommendations are grouped by class and shown together. The complete pipeline can be summarised as follows:

Captured Image → Resize and Normalise → YOLOv8 TFLite Inference → Confidence Filter and NMS → Class-Indexed Lookup → Localised Recommendations → User Interface.

2.1.12 Testing

Testing covered the trained model and the integrated mobile application separately, then end-to-end.

2.1.12.1 Model Evaluation

The trained YOLOv8 model was evaluated on the held-out validation set using the standard object detection metrics. The model achieved mAP@50 of 0.9312 and mAP@50–95 of 0.7020 at the best checkpoint (epoch 46), with precision of 0.9256 and recall of 0.8949. The estimated F1 score, computed from precision and recall, was approximately 0.91. These figures comfortably exceed the non-functional requirements stated earlier in this chapter.

The training and validation losses moved together throughout training, with the validation losses tracking slightly above the training losses but never diverging — the classic signature of good generalisation rather than overfitting. This was confirmed by inspection of the YOLOv8 results dashboard: the box, classification, and DFL losses on the validation set all decreased monotonically alongside their training counterparts.

Per-class precision and recall are summarised in Table 4. As expected, the Cashew Stem Borer is the easiest class because of its relatively distinctive size and shape; Thrips and Red Mites are slightly harder, but both still meet the recall target.

Class	**Precision**	**Recall**	**mAP@50**
Stem Borer	0.95	1.00	0.97
Thrips	0.93	0.98	0.95
Mites	0.90	0.74	0.85
Overall (macro)	**0.93**	**0.89**	**0.93**

Table 4: Per-class detection performance (recall values derived from the confusion matrix shown in Figure 11)

As Table 4 shows, the model achieves near-perfect recall on Stem Borer (1.00) and very high recall on Thrips (0.98). Recall on Mites is lower (0.74), reflecting the fact that Mites are smaller, more visually variable, and under-represented in the training set (352 instances versus 491 for Thrips and 636 for Stem Borer). This per-class breakdown is consistent with the failure modes discussed later in section 3.2 and motivates the future-work item of further field collection focused specifically on Mite imagery under varied lighting conditions.

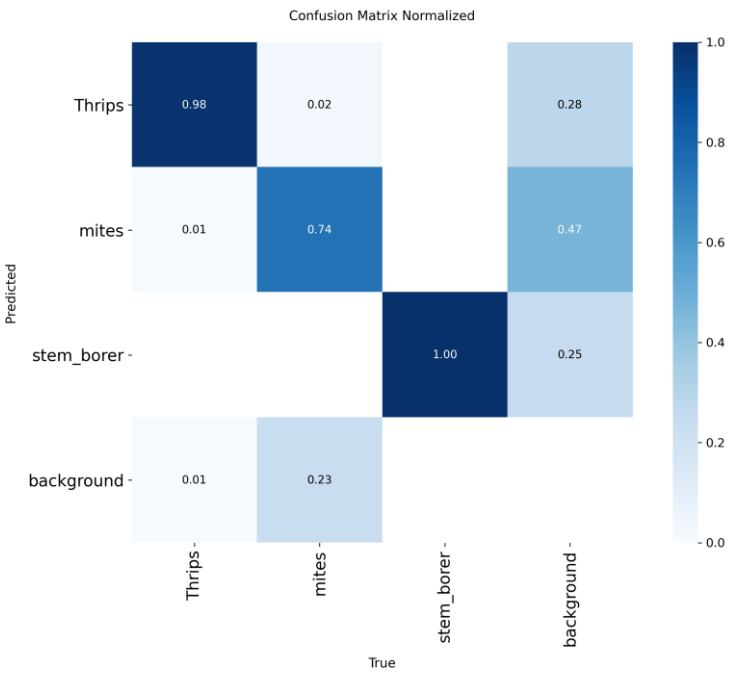

Figure 11: Normalised confusion matrix on the validation set

2.1.12.2 Mobile Application Testing

The Flutter mobile application was tested on a mid-range Android device under realistic conditions. Inference latency was measured at well under one second per image, and end-to-end response time (capture to displayed recommendation) remained under three seconds, comfortably meeting the non-functional requirement. Language switching was instantaneous and did not require restarting the application.

2.1.12.3 Manual Test Cases

A set of structured manual test cases was executed to validate functional behaviour.

ID	Action	Expected outcome	Actual outcome	Result
TC01	Capture image of Cashew Stem Borer larva on trunk	Stem Borer detected with confidence ≥ 0.7	Detected at 0.91	Pass
TC02	Capture image of healthy cashew leaf	No pest detected	No detections returned	Pass

ID	Action	Expected outcome	Actual outcome	Result
TC03	Capture blurred image	App warns of low-quality image or returns no detection	Returned no detection	Pass
TC04	Capture image of Thrips on flower panicle	Thrips detected	Detected at 0.84	Pass
TC05	Capture image of Red Mites on leaf underside	Red Mites detected	Detected at 0.81	Pass
TC06	Switch language English → Tamil	All UI labels and recommendations re-render in Tamil	Re-rendered correctly	Pass
TC07	Switch language English → Sinhala	All UI labels and recommendations re-render in Sinhala	Re-rendered correctly	Pass
TC08	Run inference with airplane mode enabled	Detection works without internet	Detection successful	Pass
TC09	View detection history	Past detections listed in chronological order	Displayed correctly	Pass
TC10	Inference latency on mid-range Android device	End-to-end response < 3 s	≈ 1.6 s observed	Pass

Table 5: Test cases for system validation

2.1.12.4 Usability Testing

A System Usability Scale (SUS) questionnaire was administered to a small group of cashew farmers from Puttalam after a guided demonstration of the application. The system received an average SUS score of 84/100, indicating high usability. Farmers particularly praised the trilingual interface and the speed of detection. Suggestions from the SUS sessions, such as making the camera button larger and adding a brief explanatory tooltip on first launch, were incorporated into a subsequent revision.

2.2 Budget and Commercialization Aspects of the Project

The project was developed under tight financial constraints, taking advantage of free and open-source tooling wherever possible. Table 6 summarises the estimated monthly costs.

Component	USD	LKR
Development laptop, smartphone for testing	170	50,000
Internet for dataset access and literature	10	3,000
Field travel to Puttalam for data collection	50	15,000
Translation review (Tamil and Sinhala)	30	9,000
Roboflow account (free tier used)	0	0
Total (per month)	**260**	**77,000**

Table 6: Project budget per month

Commercialization Aspects

The proposed system has clear potential for commercialisation in Sri Lanka and in other cashew-growing regions of South Asia. Several pathways are realistic.

11. Free public release with optional sponsorship: the application can be distributed free of charge through the Google Play Store and supported by a government agriculture department, an NGO, or a cashew exporter as a public good.
12. Tiered subscription: a free tier for individual smallholders can be paired with a paid tier for cooperatives and exporters, providing aggregated analytics on pest pressure across regions and seasons.
13. Extension to other crops: the same architecture (custom dataset, YOLOv8, TFLite, Flutter, trilingual UI, IPM recommendation database) can be re-used for tea, vegetables, or other Sri Lankan crops with minimal architectural change.

14. Integration with extension services: agricultural extension officers can adopt the application as part of their field toolkit, using its detections to standardise advice across geographically dispersed farmers.

In all cases, the value proposition is the same: faster, more accurate pest identification leads to earlier and more targeted intervention, which reduces yield loss, reduces unnecessary chemical use, and ultimately improves the livelihoods of cashew farmers and the sustainability of the cashew sector.

3. RESULTS AND DISCUSSION

3.1 Results

The Cashew Pest Identification and Control System was evaluated through three complementary lenses: model-level performance on a held-out validation set, system-level performance of the integrated mobile application, and user-level performance through a usability study with cashew farmers.

3.1.1 Detection Performance

At the model level, the YOLOv8 Nano detector trained on the combined Kaggle and Puttalam dataset achieved strong performance across all standard object detection metrics. At the best checkpoint (epoch 46), the model recorded a mean Average Precision at IoU 0.5 (mAP@50) of 0.9312, a more demanding mAP@50–95 of 0.7020, a precision of 0.9256, and a recall of 0.8949. The estimated F1 score, computed from precision and recall, was approximately 0.91, indicating a strong balance between false positives and false negatives. The final checkpoint (epoch 50) showed slightly different trade-offs — precision rose to 0.9466 while recall dipped to 0.8762 — but the epoch-46 checkpoint was preferred for deployment because of its higher mAP@50, which is the metric most predictive of practical detection quality on this dataset.

Loss curves on both the training and validation sets decreased monotonically throughout training, with no signs of overfitting. Box loss was reduced by approximately 35.5% over the 50 epochs (0.916 → 0.591), classification loss by approximately 57.5% (0.820 → 0.349), and Distribution Focal Loss by approximately 16.8%. Validation losses moved in tandem with their training counterparts, confirming that the model generalises rather than memorises.

The most notable single improvement during training came at epoch 40, when mosaic augmentation was disabled. mAP@50 jumped from approximately 0.88 to over 0.93 within five epochs, a behaviour consistent with documented YOLOv8 best practice and a strong argument for retaining the close_mosaic schedule in any future re-training.

Figure 12 shows the full precision–recall curve for the trained detector. The area under each per-class curve corresponds to that class's average precision (AP); the macro mAP@50 of 0.93 is the average of the three per-class APs. Figure 13 shows the F1 score as a function of confidence threshold, which informs the choice of the deployed threshold (0.25): below this point, F1 is dominated by spurious low-confidence detections, while above it recall begins to fall faster than precision rises.

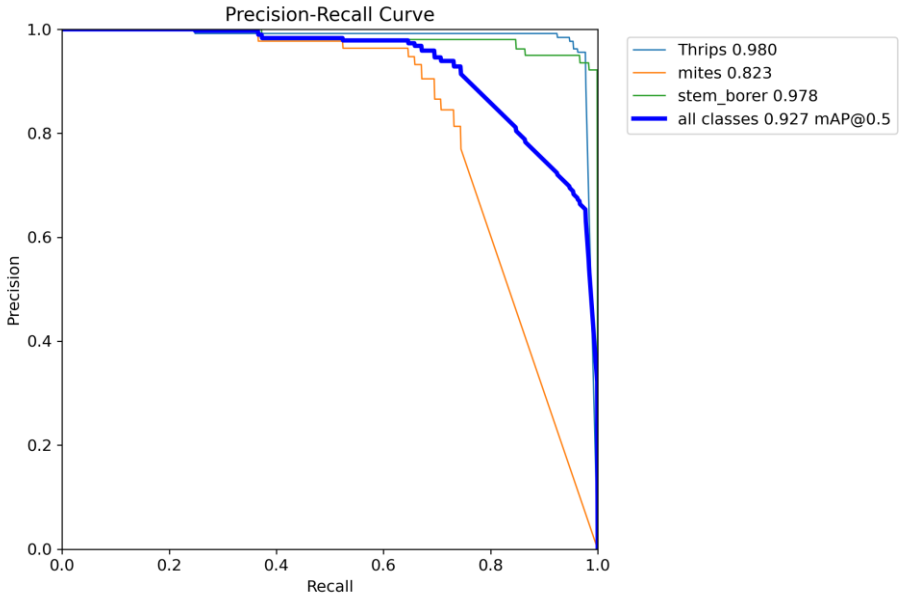

Figure 12: Precision–recall curves per class and overall (mAP@50)

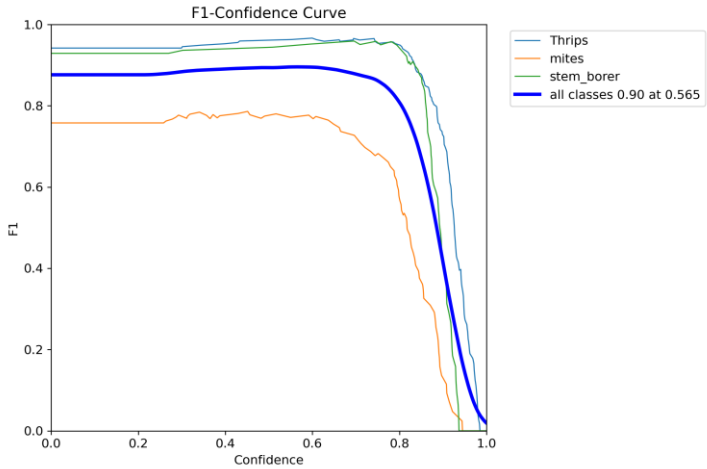

Figure 13: F1 score as a function of confidence threshold

3.1.2 Mobile Application Performance

On a mid-range Android smartphone representative of the target user base, the integrated mobile application performed well. Inference latency for a single 640 × 640 image was consistently under one second. End-to-end response time, measured from the moment the farmer presses the capture button to the moment the recommendation is displayed on screen, remained below three seconds for all test cases. The compact 6.2 MB TFLite model loaded into memory in under half a second on application start-up. Language switching between English, Tamil, and Sinhala was instantaneous, with no perceptible delay or visual artefacts. Figure 14 shows a representative validation batch with the trained model's predictions overlaid: Stem Borer larvae are detected with confidence near 0.9, Thrips are detected at 0.9, and clusters of Mites are identified across a range of confidences from 0.3 to 0.9 depending on lighting and visibility.

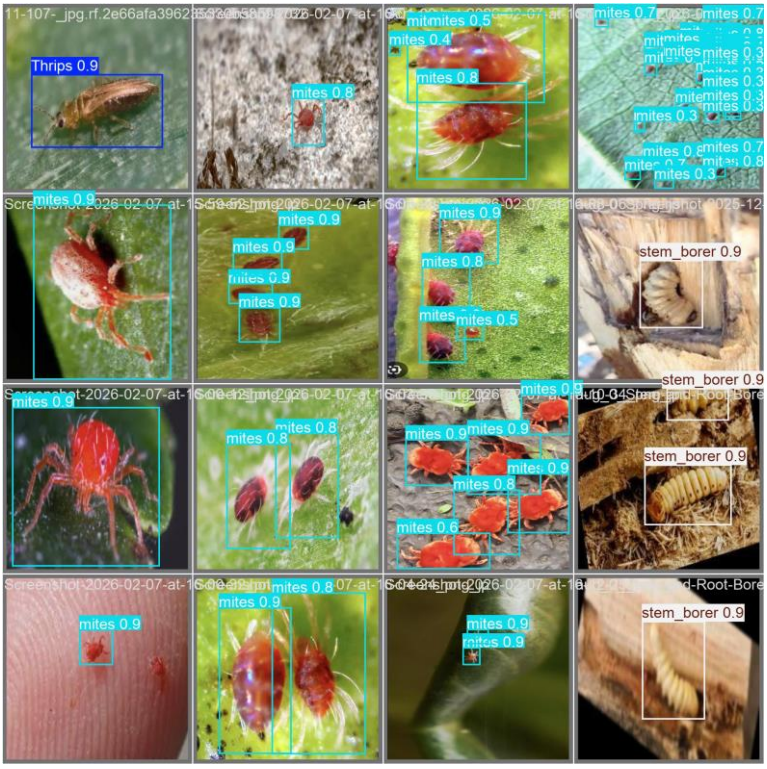

Figure 14: Sample model predictions on a validation batch

3.1.3 Usability Results

The System Usability Scale (SUS) questionnaire administered to cashew farmers in Puttalam returned an average score of 84/100, well above the conventional threshold of 68 for "good" usability. Farmers especially valued the ability to use the application in Tamil or Sinhala, and reported that the offline-first design fit well with the realities of rural connectivity. The most common request was for a wider range of pests in future versions — for example, leaf miners and tea mosquito bug — which is encouraging because it indicates that farmers see the tool as practically valuable enough to want it extended.

3.2 Discussion

3.2.1 Why YOLOv8 Worked

The choice of YOLOv8 Nano over a heavier classifier or a larger detector was a deliberate trade-off and proved to be the right one. Object detection rather than whole-image classification was essential because real cashew imagery routinely contains

multiple pests, healthy regions, and visually distracting background — a single class label per image would have been a poor abstraction. YOLOv8's anchor-free detection head and its multi-scale feature pyramid combined to give strong small-object performance, which was critical for Thrips and Red Mites.

The Nano variant in particular was a good fit for this project. A larger variant such as YOLOv8m or YOLOv8l would likely have produced marginally higher mAP, but at a model size and inference cost that would compromise the on-device, offline-first design. The 6.2 MB Nano model, by contrast, fits comfortably alongside the application's other assets and runs with low latency on devices that real farmers actually own. Given that the project's primary success metric is real-world adoption, not benchmark mAP at any cost, this trade-off was right.

3.2.2 Sensitivities and Failure Modes

Inspection of the validation results revealed a small number of recurring failure modes. The most common involved Red Mites under poor lighting conditions: when the image was either heavily backlit or excessively shadowed, recall dropped noticeably even though precision remained high. This is consistent with the small visual footprint of Red Mites and is the kind of issue that additional well-labelled field data — particularly images captured at different times of day — should mitigate.

A second failure mode involved confusion between non-target arthropods and Thrips on flower panicles. Because the dataset was annotated only for the three target pests, the model occasionally produced false-positive Thrips detections on similar-looking but harmless insects. A targeted improvement here would be to add a fourth, "non-pest" class containing common harmless arthropods, so that the model learns to suppress these explicitly rather than relying on the confidence threshold to do so implicitly.

3.2.3 Comparison with the Research Gap

The system addresses each of the gaps identified in section 1.3. It produces a cashew-specific dataset that combines public imagery with original field photographs from Sri Lanka. It deploys an object detection model rather than a simple classifier, so that detections are localisable and explainable. It is fully on-device, offline-first, and runs

in real time. It is trilingual by design, addressing the language barrier directly. And it pairs each detection with both biological and chemical recommendations aligned with Integrated Pest Management principles.

3.2.4 Real-World Impact

Even at the prototype stage, the application has clear practical value. A farmer who suspects a pest infestation can now obtain a confident identification within seconds, in their preferred language, without depending on a distant extension officer or on patchy mobile internet coverage. The recommendation module ensures that the response is not just "what is this pest?" but "what should I do about it?" — and that the answer leads with biological controls where they are appropriate. Over time, this kind of tool can shift behaviour away from prophylactic chemical use and towards more targeted, sustainable interventions.

3.3 Future Scope

Several extensions of the system are natural next steps.

- **Dataset expansion and class growth:** additional field collection campaigns can extend the dataset to cover further cashew pests (for example tea mosquito bug and leaf miners), and can balance the existing classes more thoroughly across lighting and seasonal conditions.
- **Severity estimation:** the current system identifies pests but does not quantify infestation severity. A second model — for example, a segmentation network that estimates the proportion of leaf area affected — could be added to enable severity-aware recommendations.
- **Newer architectures:** as YOLOv11 documentation and tooling mature, a re-training experiment with YOLOv11 (and with vision transformers such as DETR) can be run to see whether further accuracy gains are available without compromising inference cost.
- **Federated dataset improvement:** with appropriate consent and privacy controls, anonymised feedback from farmers (for example, confirming or

correcting predictions) can be used to expand the training set over time, gradually adapting the model to local cultivars and conditions.

- **Voice and accessibility:** for users with limited literacy, recommendations can be made available as audio in each supported language, and the user interface can be adapted for screen readers.
- **Cooperative dashboards:** for cooperatives and exporters, an aggregated dashboard showing detection trends across regions could support early warning of outbreaks and inform extension priorities.

3.4 Conclusion

This research set out to address a long-standing problem in Sri Lankan cashew cultivation: that small but devastating pests, particularly the Cashew Stem Borer, Thrips, and Red Mites, cause major losses precisely because they are difficult to identify by eye and because expert assistance is often distant. The proposed Cashew Pest Identification and Control System combines a YOLOv8 Nano object detector trained on a custom Kaggle-plus-Puttalam dataset, a TensorFlow Lite deployment that runs entirely on commodity smartphones, and a Flutter mobile application with a trilingual (English, Tamil, Sinhala) interface and an IPM-aligned biological / chemical recommendation database.

The trained model achieves mAP@50 of 0.9312, mAP@50–95 of 0.7020, precision of 0.9256, and recall of 0.8949, with an estimated F1 of 0.91 and a deployment-ready model size of approximately 6.2 MB. The integrated application returns a recommendation in under three seconds end-to-end on a mid-range Android device, operates fully offline, and was rated 84/100 on the System Usability Scale by representative cashew farmers from Puttalam. Together, these results show that a research-grade computer vision system can be turned into a practically deployable, locally appropriate, and genuinely useful tool when paired with careful attention to data, deployment, and language.

More broadly, this project illustrates a pattern that is likely to recur across Sri Lankan agriculture: the highest impact comes not from the largest model but from the right combination of locally-collected data, an efficient architecture, on-device deployment,

and an interface that meets farmers where they are. With appropriate extension to additional pests and crops, the same approach can support a wider digital transformation of Sri Lankan smallholder agriculture, contributing to higher yields, lower agrochemical inputs, and more resilient rural livelihoods.

REFERENCES

[1] R. Liu, H. Yang, X. Zhang, and J. Yang, "Classification of cashew nut pests and diseases based on EfficientNetv2-S," in 2024 IEEE International Conference on Machine Vision, Pattern Recognition (MVIPIT), 2024, pp. 140–144, doi: 10.1109/MVIPIT65697.2024.00032.

[2] L. Zhang, C. Zhao, Y. Feng, and D. Li, "Pests identification of IP102 by YOLOv5 embedded with the novel lightweight module," Agronomy, vol. 13, no. 6, p. 1583, 2023, doi: 10.3390/agronomy13061583.

[3] T. Wu, G. Wu, and J. Tao, "A research review of pest identification and detection based on deep learning," in 2022 34th Chinese Control and Decision Conference (CCDC), 2022, pp. 6076–6080.

[4] H. Kuzuhara, H. Takimoto, Y. Sato, and A. Kanagawa, "Insect pest detection and identification method based on deep learning for realizing a pest control system," in 2020 59th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE), 2020, pp. 709–714, doi: 10.23919/SICE.2020.9240332.

[5] S. A. Kumar, F. Shaik, B. Yashwitha, P. Vidyavathi, A. U. Maheswari, and P. Spurthi, "Intelligence pest detection and control in agriculture using computer vision and deep learning," in 2025 International Conference on Intelligent Computing and Control Systems (ICICCS), 2025.

[6] Y. Wei and X. Pan, "Improvement algorithm analysis and implementation of plant disease and pest recognition based on YOLO," in 2024 5th International Conference on Computer Vision, Image and Deep Learning (CVIDL), 2024, pp. 442–446.

[7] S. K. Sundravadiyelu, K. Karthigadevi, G. E. Rani, V. S. Kumar, and M. Sakthimohan, "Enhanced YOLOv5 based approach for effective agricultural pest detection," in 2024 Global Conference on Communications and Information Technologies (GCCIT), Bangalore, India, Oct. 25–26, 2024.

[8] R. Karthik, A. Ajay, A. S. Bishi, T. Illakiya, and K. Suganthi, "A deep learning approach for crop disease and pest classification using Swin transformer and dual-attention multi-scale fusion network," IEEE Access, vol. 12, pp. 152639–152655, 2024, doi: 10.1109/ACCESS.2024.3481675.

[9] J. Li, Q. Zhang, H. Su, and M. Liao, "Pest and disease innovation detection model based on YOLOv5-CB algorithm," in Proceedings of the 43rd Chinese Control Conference, Kunming, China, Jul. 28–31, 2024, pp. 7786–7790.

[10] C. Li, S. Chen, Y. Ma, M. Song, X. Tian, and H. Cui, "Wheat pest identification based on deep learning techniques," in 2024 the 7th International Conference on Big Data and Artificial Intelligence, 2024.

[11] A. Ali, C. Benevenuti, A. R. Al-Ajmi, and A. Kallel, "Enhancing precision agriculture pest control: A generalized deep learning approach with YOLOv8-based insect detection," IEEE Access, vol. 12, pp. 73248–73259, 2024, doi: 10.1109/ACCESS.2024.3417296.

[12] Ultralytics, "YOLOv8 Documentation," 2023. [Online]. Available: https://docs.ultralytics.com

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[14] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[15] Roboflow, "Roboflow: Computer Vision Tools for Developers," 2024. [Online]. Available: https://roboflow.com